

Final Report

Sparse Matrices Analysis and Applications

Hesham Ashraf 202201477

Ahmed Wael 202201415

Abstract

Sparse matrices, which contain many zero elements, play an important role in reducing memory usage and computational cost in large-scale numerical problems. This project aims to analyze sparse matrix structures and investigate efficient storage techniques and numerical methods for solving systems of linear equations. Different approaches, including sparse matrix representations and iterative solution methods, will be implemented and compared. Simulations will be conducted using Python in Google Colab to evaluate performance, accuracy, and computational efficiency. The effect of matrix size and sparsity level on the overall performance will also be studied.

Introduction

In many scientific and engineering applications, large systems of linear equations arise that involve matrices with a significant number of zero elements. These matrices are referred to as sparse matrices. Unlike dense matrices, sparse matrices allow for more efficient storage and computation by only storing non-zero elements.

Sparse matrix techniques are widely used in fields such as computational physics, engineering simulations, and data science. Efficient handling of sparse matrices can significantly reduce both memory consumption and execution time. Various numerical methods have been developed to solve sparse systems, including direct methods and iterative methods such as Jacobi and Gauss-Seidel.

This project aims to investigate the performance and efficiency of sparse matrix techniques by implementing and comparing different storage formats, including CSR (Compressed Sparse Row), CSC (Compressed Sparse Column), and dense matrix representations. The project also focuses on developing iterative solvers, specifically the Jacobi and Gauss-Seidel methods, to solve systems of linear equations efficiently. In addition, benchmarking will be conducted to evaluate memory consumption, execution time, and error convergence for each approach. Finally, the project will analyze scalability by examining how varying sparsity levels and increasing matrix sizes affect overall computational performance.

Problem Definition

The problem is to solve a system of linear equations of the form:

$$Ax=b$$

where:

- A is a sparse matrix of size (n, n)
- x is the unknown vector
- b is a known vector

The goal is to:

- Efficiently store matrix A using sparse representations
- Solve the system using numerical methods.
- Compare performance with dense matrix approaches.

Methodology

Using the following Numerical Methods

1. Jacobi Iteration Method

The Jacobi method decomposes the matrix A into:

$$A = D + (L + U)$$

where D represents the diagonal matrix, while L and U are the strict lower and upper triangular matrices, respectively. The iterative update formula is:

$$x^{(k+1)} = D^{-1}(b - (L + U)x^{(k)})$$

The method computes each variable independently using values from the previous iteration, making it suitable for parallel computation.

```
def jacobi_solver(A, b, x0, max_iter=1000, tol=1e-6):
    n = A.shape[0]
    x = x0.copy()
    errors = []

    # Get diagonal elements
    D = A.diagonal()
    if np.any(D == 0):
        raise ValueError("Zero on diagonal! Jacobi cannot proceed.")

    start_time = time.time()
    for i in range(max_iter):
        r = b - A.dot(x)
        x_new = x + r / D

        err = np.linalg.norm(x_new - x)
        errors.append(err)
        x = x_new

        if err < tol:
            break

    return x, i+1, time.time() - start_time, errors
```

2. Gauss-Seidel Method

The Gauss-Seidel method improves upon the Jacobi approach by immediately using newly computed values during the iteration process. Its update formula is:

$$x^{(k+1)} = (D + L)^{-1}(b - Ux^{(k)})$$

Because updated values are reused within the same iteration, the Gauss-Seidel method generally converges faster than the Jacobi method for many systems.

3. Direct Solver

Using `scipy.sparse.linalg.spsolve`, which typically uses a variant of LU factorization (SuperLU) optimized for sparse structures.

```
def gauss_seidel_solver(A, b, x0, max_iter=1000, tol=1e-6):
    n = A.shape[0]
    x = x0.copy()
    errors = []

    A_csr = A.tocsr()
    start_time = time.time()

    for k in range(max_iter):
        x_old = x.copy()
        for i in range(n):
            row_start = A_csr.indptr[i]
            row_end = A_csr.indptr[i+1]

            cols = A_csr.indices[row_start:row_end]
            vals = A_csr.data[row_start:row_end]

            dot_product = np.dot(vals, x[cols])
            diag_val = A_csr[i, i]

            x[i] = (b[i] - (dot_product - diag_val * x[i])) / diag_val

        err = np.linalg.norm(x - x_old)
        errors.append(err)
        if err < tol:
            break

    return x, k+1, time.time() - start_time, errors
```

Performance Comparison

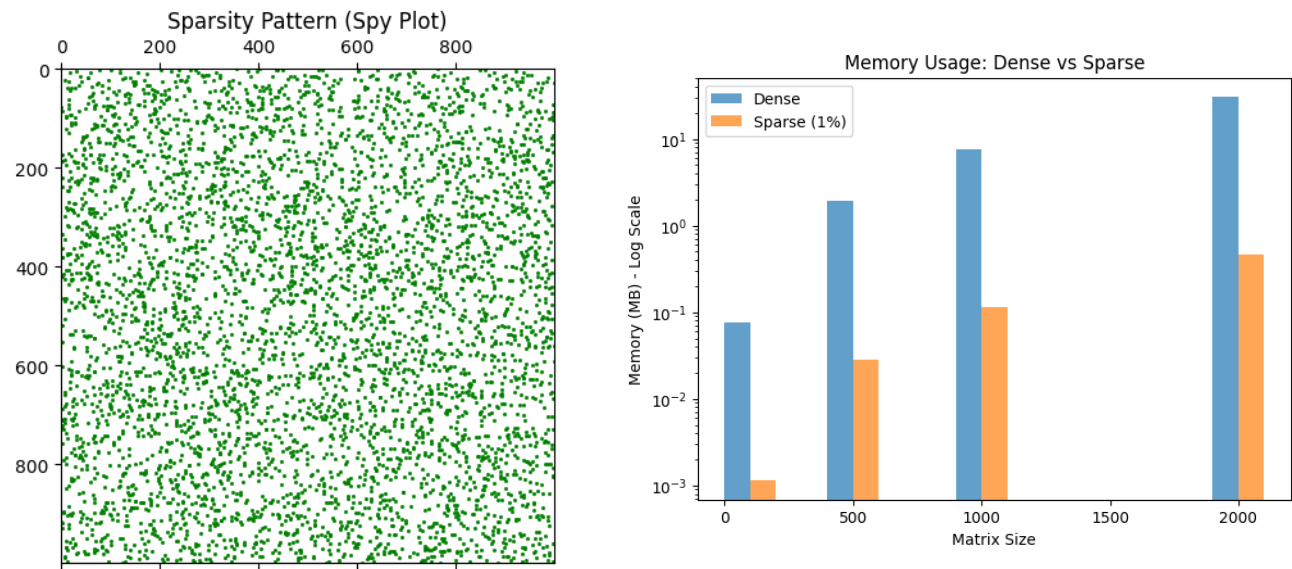
To evaluate the efficiency of the implemented methods, a performance comparison was conducted between the Jacobi method, Gauss-Seidel method, and the direct sparse solver. The comparison focused on two main metrics: execution time and the number of iterations required for convergence. Execution time was measured to determine the computational efficiency of each method as the matrix size increased, while iteration counts were used to analyze convergence speed. The results provide insight into the trade-offs between iterative and direct solution techniques, particularly in handling large sparse matrices. In general, the Gauss-Seidel method is expected to converge in fewer iterations than the Jacobi method because it utilizes newly updated values immediately, whereas the direct solver typically achieves the solution faster for smaller systems but may require higher memory usage for large-scale problems.

Results

The experimental results demonstrate the significant advantages of sparse matrix representations in terms of memory efficiency and computational performance. As the matrix size n increases, dense matrix storage grows quadratically, with a complexity of $O(n^2)$, leading to substantial memory consumption for large systems. In contrast, sparse storage formats such as CSR and CSC

scale according to the number of non-zero elements, $O(nnz)$, resulting in much lower memory usage when the matrix contains many zero entries.

	Matrix Size	Non-Zero Elements	Sparsity %	Dense Memory (MB)	CSR Memory (MB)	CSC Memory (MB)
0	1000x1000	5000	99.5	7.629395	0.061039	0.061039



The solver comparison also revealed clear differences in computational behavior between iterative and direct methods. The Jacobi and Gauss-Seidel iterative methods showed performance that was highly dependent on the matrix structure and sparsity pattern. Among the iterative techniques, Gauss-Seidel generally converged in fewer iterations because it immediately reused updated values during computation, while the Jacobi method relied entirely on values from the previous iteration. On the other hand, SciPy’s direct sparse solver achieved very fast solution times for the tested matrix sizes due to its optimized sparse LU factorization approach.

	Matrix Size	Jacobi Time	Jacobi Iter	Gauss-Seidel Time	Gauss-Seidel Iter	Direct method Time
0	100	0.001658	13	0.015636	9	0.001235
1	300	0.000269	25	0.052268	11	0.001330
2	500	0.000512	39	0.080972	11	0.004910
3	800	0.000872	60	0.126198	10	0.029046

Conclusion

This project successfully demonstrated the implementation and analysis of sparse matrices in Python. The key findings show that sparse formats such as CSR and CSC significantly reduce

memory overhead compared to dense representations, especially for large systems, achieving substantial memory optimization. Overall, the study confirms that using sparse matrix representations together with appropriate numerical solvers greatly improves both computational efficiency and scalability when handling large linear systems. The applications of these methods are wide-ranging, including engineering tasks such as the Finite Element Method (FEM) for structural analysis, physics simulations in fluid dynamics and quantum mechanics, data science applications like recommendation systems and Natural Language Processing, as well as machine learning domains involving large-scale neural network weights and graph convolutional networks.

References

Scott, J., & Tuma, M. (2023). An introduction to sparse matrices. In *Algorithms for Sparse Linear Systems* (pp. 1-18). Cham: Springer International Publishing.

Zhang, X. (2017). *Matrix analysis and applications*. Cambridge University Press.

Rose, D. (Ed.). (2012). *Sparse Matrices and their Applications: Proceedings of a Symposium on Sparse Matrices and Their Applications, held September 9–10, 1971, at the IBM Thomas J. Watson Research Center, Yorktown Heights, New York, and sponsored by the Office of Naval Research, the National Science Foundation, IBM World Trade Corporation, and the IBM Research Mathematical Sciences Department*. Springer Science & Business Media.